4-4 · Trino 기반 S3 SQL 엔진 구축 · 실습 1H

Trino SQL 실습

배운 SQL 이 도구를 바꿔도 그대로 통하는지

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① 고객별·채널별 집계와 윈도 함수를 Trino 에서 실행한다

② CTAS 로 daily_sales 를 만들고 Spark 에서도 읽히는지 확인한다

③ "orders$snapshots" 와 FOR VERSION AS OF 로 타임 트래블한다

④ EXPLAIN·웹 UI 를 보고, Trino 가 쓴 파일을 Spark 로 읽어 본다

쓰기는 되지만 대량 변환은 여전히 Spark — 역할을 기억하며 실습합니다

실습 흐름

여섯 단계 — 읽고, 만들고, 되돌아가고, 써 본다

Trino CLI(docker compose exec -it trino trino)에 접속한 상태에서 시작합니다

1. 집계	2. CTAS	3. 타임 트래블	4. 실행 계획	5·6. 쓰기·과제
GROUP BY 윈도 함수	daily_sales Spark 로 확인	$snapshots AS OF 조회	EXPLAIN 웹 UI 8080	Spark 로 읽기 채널별 비중

1단계

세 쿼리 — 2-2·3-4 의 SQL 그대로

1. **고객별 합계**
   GROUP BY customer → 6행, choi 61000.0 이 맨 위

2. **채널별 · 날짜별**
   GROUP BY order_date, channel → 5행, 08-03 app 2건 79000.0

3. **누적 합계(윈도 함수)**
   sum(amount) OVER (ORDER BY …) → 15000.0 에서 165000.0 까지 6행

확인 포인트

세 쿼리

모두 동작

> **FROM iceberg.lakehouse.orders — 도구를 또 바꿔도 SQL 은 그대로 통한다. SQL 을 익혀 두는 것의 가치**

2단계

CTAS 로 만든 표를 Spark 가 읽는가

1. **CREATE TABLE ... AS SELECT**
   daily_sales — order_date 별 order_cnt·revenue → 3 rows

2. **Trino 에서 조회**
   08-01 1 15000.0 · 08-02 2 47000.0 · 08-03 3 103000.0

3. **Spark 에서 같은 표**
   두 번째 터미널 셸에서 cd ~/lakehouse → spark-sql — 같은 세 줄

4. **MinIO 에 파일이 생겼나**
   lakehouse/daily_sales-<고유 문자열>/ 아래 data·metadata

성공 조건

양쪽 같은 세 줄

+ 파일 생성

3단계

타임 트래블 — Spark 와 Trino 문법 차이

하려는 일	Spark	Trino
스냅샷 이력	FROM t.snapshots	FROM "t$snapshots"
파일 목록	FROM t.files	FROM "t$files"
버전 조회	VERSION AS OF id	FOR VERSION AS OF id
시각 조회	TIMESTAMP AS OF '...'	FOR TIMESTAMP AS OF TIMESTAMP '...'
테이블 이름	demo.lakehouse.orders	iceberg.lakehouse.orders

첫 snapshot_id 로 FOR VERSION AS OF → 4-2 직후 네 행 · 숫자를 안 바꾸면 Iceberg snapshot ID does not exists

따옴표 없이 orders$snapshots 를 치면 mismatched input '$' — 일부러 한 번 쳐 보세요

4단계

EXPLAIN 과 웹 UI 로 실행 계획 보기

1. **Fragment 1 [SOURCE] — 아래 조각**
   TableScan 으로 읽고 Aggregate(PARTIAL)로 미리 합친다
2. **Fragment 0 [HASH] — 위 조각**
   RemoteSource · LocalExchange → Aggregate(FINAL) → Output
3. **http://localhost:8080 (/ui/)**
   이름은 아무 값으로 로그인 — 쿼리를 클릭하면 단계별 처리 행 수
4. **느린 쿼리를 만나면**
   어느 단계가 오래 걸렸는지 여기서 — 5주차 Spark UI 도 비슷

5단계 · 일부러 실패시키기

Trino 로 쓴 표를 Spark 가 읽을 수 있을까

1. **Trino 로 INSERT · DELETE**
   order_id 99 추가 → 1 row · 7, 삭제 → 1 row · 6

2. **셀에서 Spark SELECT ***
   Cannot support vectorized reads … DELTA_LENGTH_BYTE_ARRAY

3. **원인**
   Trino 483 의 글자 컬럼 형식을 Iceberg 1.8.1 빠른 읽기가 못 읽는다

4. **테이블 속성으로 끄기**
   ALTER TABLE … SET TBLPROPERTIES → SELECT * 여섯 행

> **'read.parquet.vectorization.enabled'='false' — count(*) 는 지나가서 놓치기 쉽다 · 4-5 를 위해 그대로 둔다**

6단계

미니 과제 — 채널별 비중과 파일 수

과제 1	채널별 매출 비중(%) → app 91000.0 55.2 · web 74000.0 44.8
관용구	sum(sum(amount)) OVER () — 집계한 값들의 전체 합을 각 행에
과제 2	daily_sales 를 Trino 에서 갱신 → Spark 에서 확인
과제 3	"orders$files" 로 데이터 파일 수와 각 행 수

round(sum(amount) * 100.0 / sum(sum(amount)) OVER (), 1) AS pct — 두 비중의 합이 100 이면 성공

확인 포인트

이 1시간이 끝날 때 확인할 것

1. 집계 세 쿼리 — 고객별 6행 · 날짜·채널 5행 · 누적 6행
2. daily_sales 세 줄이 Spark 에서도 같고 폴더가 생겼다
3. "orders$snapshots" 이력이 같고 첫 id 로 네 행이 보인다
4. INSERT 뒤 7·DELETE 뒤 6 — Spark 는 벡터화 읽기 실패 확인
5. 속성을 끈 뒤 Spark SELECT * 가 여섯 행을 낸다
6. 채널별 비중 app 55.2 · web 44.8 — 합이 100

4-4 · 실습 1H 마무리

다음은 정리·회고 (0.5H)

두 엔진의 문법 차이를 정리하고, 4-5 통합 검증을 위한 상태를 확인합니다

- ✓ 오늘 확인한 것 — 양방향으로 통한다
- ✓ 컨테이너 5개 Up · 두 엔진 count 6
- ✓ 자주 겪는 오류 네 가지
- ✓ 4-5 카탈로그 연결·레이크하우스 통합 예고

가져갈 것

orders·daily_sales 두 표가 남은 상태